

Decision Brief: NEON Site Compatibility

ExSOIL NSF Prototype · June 2026 · Steven Wangen

Open for Discussion

Executive Summary

The project's Docker container was recently rebuilt from scratch on Ubuntu 24.04 with native support for both Intel/AMD (amd64) and Apple Silicon (arm64) processors. The multi-arch infrastructure, conda environment, 89-test validation suite, and CESM Fortran build system are all working. What is missing is the **NEON tower site workflow**, the project's primary use case.

The previous container (`escomp/cesm-lab-neon`) was an amd64-only, custom image that mixed the CESM 2.2 framework with a newer CTSM development branch to get NEON support. This was a one-off build by an NCAR developer, not part of any official CESM release. **NEON support was never part of the CESM 2.x development pipeline.** It was developed exclusively on CTSM's master branch starting in February 2021, and will only enter CESM when version 3.x ships (currently in alpha).

Our rebuild used the official CESM 2.2.2 release, which is clean and reproducible but does not include NEON because the 2.2 branch predates that work entirely. We did not lose something that was there before; the old container was a non-standard assembly.

This brief presents five options for restoring NEON functionality:

Option	Approach	Effort	NEON available now?
A-C	Bolt NEON onto our existing CESM 2.2.2 container (graft config files, upgrade just CLM, or maintain our own site configs)	Low to medium-high	Yes, with validation risk
D	Upgrade to CESM 3.x (full coupled Earth system with NEON natively)	High	No (CESM 3.x not yet released)
E	Drop CESM, use standalone CTSM (what NCAR built the NEON workflow for)	Medium	Yes

The multi-arch Dockerfile, conda environment, and test framework we built carry over to any option. The choice depends on the project's scope, timeline, and whether fully coupled atmosphere-land-ocean simulations are on the roadmap. Option E trades away coupled simulation capability (which the project does not currently use) for a cleaner, smaller, NCAR-recommended path to NEON support.

Background: How CESM, CLM, and NEON Fit Together

Before diving into the problem, it helps to understand the system architecture. **CESM** (Community Earth System Model) is a modular climate simulation framework maintained by NCAR. It models the entire Earth system by coupling separate components that each handle a different domain:

Component	Domain	What it simulates
CAM	Atmosphere	Weather, radiation, clouds, precipitation
CLM	Land surface	Soil temperature/moisture, vegetation, carbon cycling, snow, rivers
POP	Ocean	Ocean circulation, heat transport, biogeochemistry
CICE	Sea ice	Ice formation, melting, drift
CISM	Ice sheets	Glaciers, Greenland/Antarctic ice dynamics
MOSART	Rivers	Freshwater routing from land to ocean

These components exchange energy, water, and momentum through a central coupler at each time step. For a full simulation you run them all together, but for land-focused research (like this project) you can run CLM by itself with prescribed atmospheric forcing data, which is much cheaper computationally.

CLM and CTSM

CLM (Community Land Model) is the component most relevant to this project. It simulates what happens at and below the land surface: how soil heats and cools, how water moves through soil layers, how plants grow and exchange carbon with the atmosphere, and how snow accumulates and melts. **CTSM** (Community Terrestrial Systems Model) is the standalone version of CLM, maintained in its own Git repository. The names are often used interchangeably; technically

CLM is the component within CESM and CTSM is the broader standalone project that contains it.

NEON and the tower workflow

NEON (National Ecological Observatory Network) operates 81 instrumented field sites across the United States. Each site has a tower equipped with sensors that continuously measure things like air temperature, humidity, wind speed, soil temperature at multiple depths, soil moisture, and carbon dioxide fluxes between the land and atmosphere. These measurements provide ground truth for testing whether CLM's predictions match reality at specific locations.

The NEON tower workflow connects CLM to these real-world observations. The idea is: take CLM, configure it for a specific NEON tower site (telling it the exact grid cell, soil properties, vegetation type, and local conditions), feed it observed meteorological data from that tower as atmospheric forcing, run the simulation, then compare CLM's predicted soil temperatures, moisture, and carbon fluxes against what the tower actually measured. This is the primary use case for this project's notebooks.

CIME and usermods

CIME (Common Infrastructure for Modeling the Earth) is the build and case management system that orchestrates all of this. When you run commands like `create_newcase` , `case.setup` , and `case.build` , you are using CIME. It handles Fortran compilation, namelist generation, MPI configuration, and job submission.

Usermods are per-site configuration bundles that tell CIME how to set up CLM for a specific NEON tower location. Each site's usermods directory (e.g., `NEON/ABBY/` , `NEON/BART/`) contains files that specify the grid cell coordinates, surface dataset paths, and namelist overrides for that site. Without usermods, you would have to manually configure all of these details every time you create a case for a NEON site.

History: When and where NEON support was developed

The NEON tower workflow was **never part of any CESM 2.x release**. It was developed specifically for standalone CTSM, on a separate development branch, after the CESM 2.2 release line had already been cut.

CESM / CTSM VERSION LINEAGE



CESM 2.2 released

Pins CLM to `release-cesm2.2.04` (from `release-clm5.0` line)

No NEON support. Never will have it on this branch.

Feb 2021 — NEON usermods development begins

On CTSM `master` branch (`ctsm5.1.dev` line), *not* the `clm5.0` release line

PR [#1278](#): "Adds support for NEON tower data sites." Merged May 2021.

2021-2022 — Active NEON development on CTSM master

Surface datasets, batch wrapper, UI improvements (PRs [#1375](#), [#1444](#), [#1467](#), [#1474](#), [#1539](#))

~2022 — `escomp/cesm-lab-neon` Docker image published

Custom build by NCAR developer. Mixed CESM 2.2 framework with a newer CTSM dev branch that had NEON support. Not built from the ESCOMP-Containers repo. No record of which exact CTSM tag was used.

Nov 2022 — First NEON-tagged CTSM release (`ctsm5.1.dev114`)

"NEON release: Some NEON updates fixing AG sites, update MOSART, small fixes."

Jun 2026 — Our multi-arch rebuild

Used clean CESM 2.2.2 checkout. Its CLM (`release-cesm2.2.04`) is on the `release-clm5.0` line, which never received NEON support.

Future — CESM 3.x (in alpha)

Will incorporate CTSM master, including NEON support natively. Not yet released.

release-clm5.0 line

`release-cesm2.2.04` → ... → `release-clm5.0.37`
(May 2024)

Maintenance patches only. No NEON. No new features.

This is what CESM 2.2.2 uses.

CTSM master (ctsm5.x)

`ctsm5.1.dev` → `ctsm5.2` → current

Active development. NEON support since Feb 2021.

This feeds into standalone CTSM and future CESM 3.x.

There is no evidence of plans to backport NEON support to CESM 2.x. NCAR's [official CTSM documentation](#) and the [NCAR CTSM Tutorial](#) (including a `NEON_Tutorial_2023` branch) frame NEON exclusively as a standalone CTSM workflow.

How the original container worked

The original `escomp/cesm-lab-neon` Docker image (published on Docker Hub by NCAR developer `bdobbins` around 2022, 1.72 GB, amd64-only) was a **custom assembly** that does not correspond to any standard CESM release. It was not built from the [ESCOMP-Containers](#) repository (which has no NEON references). It appears to have been built separately using a development CTSM branch with NEON support, layered onto a CESM 2.2-era framework. NCAR's [ncar-neon-books](#) documentation references the image but does not document which CTSM tag was used inside.

What changed in the rebuild

When we rebuilt the container for multi-architecture support, we replaced the old custom ESCOMP image with a clean CESM 2.2.2 checkout from GitHub. This is a reproducible, officially tagged release, but its CLM component (`release-cesm2.2.04`) is part of the `release-clm5.0` line that predates NEON development entirely.

Key insight

We did not lose something that was there before. CESM 2.2 never had NEON support. The old container was a non-standard build that mixed components from different version lines. Our rebuild used a clean checkout, which is better for maintainability but does not include the NEON customization that was grafted onto the old image. This strengthens the case for **Option E (standalone CTSM)**, since that is how NCAR actually intends NEON to be used.

Problem

The NEON site workflow is non-functional in the rebuilt container.

CESM 2.2.2's CLM component predates the NEON tower site workflow. The per-site configuration directories ("usermods") that `run_neon_v2.py` depends on do not exist in this CLM release. Running any NEON site command fails immediately:

```
$ run_neon_v2 --neon-sites ABBY --output-root ~/output --overwrite
```

```
run_neon_v2: error: argument --neon-sites: invalid choice: 'ABBY' (choose from 'all')
```

Why this happens

The script discovers available sites by scanning a directory in the CLM source tree:

```
valid_neon_sites = sorted([
    v.split("/")[-1]
    for v in glob.glob(
        os.path.join(cesmroot, "cime_config", "usermods_dirs", "NEON", "[!d]*")
    )
])
```

In the original `escomp/cesm-lab-neon` container (built ~2022), this directory contained 81 NEON site folders (ABBY, BART, KONZ, etc.). In CESM 2.2.2's standard CLM checkout, **this directory does not exist**. The NEON usermods were added to CTSM after the 2.2.x release series.

What still works

Capability	Status	Notes
General CESM case creation	Working	I2000CIm50Sp, B1850, etc. all verified
CESM Fortran compilation	Working	case.build produces cesm.exe in 78s on arm64
Python scientific stack	Working	All 28 tutorial packages, 63 smoke tests pass
JupyterLab	Working	Starts, serves notebooks, Panel extensions loaded
NEON site runs (run_neon_v2)	Broken	No valid site codes; usermods directory missing
Design_Hub / Modeling_Hub notebooks	Broken	Depend on NEON site runs

Options

Option A: Graft NEON usermods from newer CTSM

Low effort

Moderate risk

Copy the `usermods_dirs/NEON/` directory from a recent CTSM release (e.g., `ctsm5.2`) into the CESM 2.2.2 CLM tree during the Docker build. This is likely how the original ESCOMP image was assembled.

Pros:

- Minimal disruption to the rest of the CESM source tree
- NEON usermods are mostly configuration files (XML, shell scripts), not compiled code
- A few `COPY` lines in the Dockerfile

Cons:

- Usermods may reference namelist options or surface datasets that don't exist in CESM 2.2.2's CLM
- Compatibility not guaranteed; could cause subtle configuration errors
- Creates ongoing maintenance burden tracking which CTSM version usermods came from

Timeline: Hours to implement, days to validate.

Option B: Pin CLM component to newer CTSM tag

Medium effort

Moderate risk

Override just the CLM/CTSM component in the CESM checkout to use a newer tag (e.g., `ctsm5.2.005`) while keeping the rest of CESM at 2.2.2. CESM's `Externals.cfg` supports per-component tag overrides.

Pros:

- Gets the full NEON workflow as designed: usermods, surface datasets, code fixes
- Officially supported CLM version with community testing
- Forward-compatible with future NEON site additions

Cons:

- Newer CTSM may have API incompatibilities with CESM 2.2.2's CIME or coupler
- Could break the case.build compilation (89/89 tests currently pass)
- Mixing component versions is officially unsupported by ESCOMP

Timeline: Days. Requires rebuild, recompile, full re-test.

Option C: Bundle NEON configs in this repository

Medium-high effort

Low risk

Create a `neon-sites/` directory in this repository with per-site usermods (shell_commands, user_nl_clm, etc.) and copy them into the CLM tree during the Docker build. Maintain independently of any CTSM release.

Pros:

- Full control over site configurations; no external version dependency
- Can limit to the specific sites the project uses (not all 81)
- Easy to add custom sites or project-specific modifications

Cons:

- Significant upfront effort to create or port usermods for each site
- Must manually maintain surface dataset references and domain file paths
- Diverges from the community CTSM NEON workflow

Timeline: Days to a week, depending on number of sites needed.

Option D: Upgrade to CESM 3.x / CTSM 6.x

High effort

High risk

Move to a current CESM release that natively includes the NEON workflow. Resolves this issue at its root and also fixes the Python 3.11 pin, the PIO2 filter patches, and other 2.2.x limitations.

Pros:

- Resolves NEON, Python version, PIO2, and other 2.2.x issues in one move
- Aligns with the community's current supported version
- Future-proofs the container for years

Cons:

- Major version upgrade with potentially breaking changes across all components
- `run_neon_v2.py` was written against 2.2.x APIs; likely needs significant rewrite
- All 89 container tests need re-validation
- Machine configurations and compiler flags may need rework

Timeline: Weeks. Essentially a second rebuild effort.

Option E: Replace CESM with standalone CTSM

Medium effort

Moderate risk

Drop the full CESM checkout and replace it with a standalone CTSM checkout. CTSM has its own `Externals.cfg` that pulls in only what land-surface modeling needs: CIME, DATM, MOSART, and stub components. It does not pull in CAM, POP, CICE, CISM, or WW3. Current CTSM releases include the NEON tower workflow natively.

This reframes the question: instead of "how do we get NEON support into CESM 2.2.2," it asks **"do we actually need CESM, or is CTSM sufficient?"**

What CTSM standalone includes:

Component	Purpose	Needed?
CLM/CTSM	Land model (soil, vegetation, carbon)	Yes (the core)
CIME	Build system, case management	Yes
DATM	Data atmosphere (feeds observed weather to CLM)	Yes
MOSART	River routing	Yes
Stubs	Placeholder ocean/ice/wave	Yes (required by coupler)
FATES	Vegetation demographics (optional)	Possibly for future work

What would be lost:

Component	Impact of losing it
CAM (atmosphere)	None for current workflows. The project uses observed forcing data, not a simulated atmosphere.
POP (ocean)	None. No ocean modeling in the project. NEON workflow uses a stub ocean.
CICE (sea ice)	None. Not relevant to land surface research at tower sites.
CISM (ice sheets)	None. Glacier/ice sheet modeling is outside scope.
Fully coupled simulations	This is the significant loss. Without full CESM, you cannot run experiments where land surface changes (deforestation, irrigation) feed back into the atmosphere and alter precipitation. The land model receives prescribed weather and cannot influence it. For the current NEON workflow this is fine. But if the project ever expands to land-atmosphere feedbacks, regional climate impacts, or paleoclimate scenarios, you would need full CESM coupling.

The project currently runs CLM in "I-compset" mode (land-only with data atmosphere), which is exactly what standalone CTSM is designed for. The full CESM components are present in the current container but never compiled or executed. They consume ~3-4 GB of source code for no functional benefit.

Scope implication

Moving to CTSM standalone is a statement about the project's scope: "we are a land surface modeling project, not a coupled climate modeling project." If that scope might expand in the future, keeping full CESM preserves optionality even if unused today.

Pros:

- NEON tower workflow included natively; this is what CTSM is built for
- Dramatically smaller image (~3-4 GB less source code)
- Current Python support (newer CIME works with 3.12+); no 3.11 pin
- Newer PIO and CIME; source patches and GCC workarounds may be unnecessary
- Aligns with how NCAR recommends running NEON tower experiments
- Simpler dependency tree; fewer components = fewer version conflicts

Cons:

- Loses ability to run fully coupled simulations if project scope expands
- `run_neon_v2.py` path assumptions need updating for CTSM directory layout
- Machine configs need porting to CTSM's version of CIME
- GCC compatibility flags may or may not still be needed; requires testing
- 89-test suite needs re-validation

Timeline: Days. Comparable to Option B, but with a cleaner result. The Dockerfile stages and conda environment carry over unchanged; only the CESM clone stage and machine configs need rework.

Comparison

	A: Graft usermods	B: Newer CLM tag	C: Bundle in repo	D: CESM 3.x	E: CTSM standalone
Effort	Low	Medium	Medium-high	High	Medium
NEON sites work	Likely	Yes	Yes	Yes	Yes
Existing tests pass	Yes	Unknown	Yes	Re-validate	Re-validate
Fixes Python 3.11 pin	No	No	No	Yes	Likely
Fixes PIO2 patches	No	No	No	Yes	Likely
Community-maintained	Partial	Yes	No	Yes	Yes
Risk to current build	Low	Moderate	Low	High	Moderate
Image size reduction	None	None	None	None	~3-4 GB smaller
Coupled simulations	Preserved	Preserved	Preserved	Preserved	Lost

Questions for Discussion

Q1 Which NEON sites does the project actively use? If it's 3-5 sites, Option C is attractive. If many sites or all 81, Options A, B, or E scale better.

Q2 How tightly coupled is `run_neon_v2.py` to CESM 2.2.x internals? If it mostly uses CIME's public API (`create_newcase`, `xmlchange`), Options B or E are safer. If it patches internal CIME files, it's riskier.

Q3 Is there appetite for a CESM 3.x migration? If the project plans to upgrade eventually, Option D avoids a throwaway step. Otherwise, a lighter option makes more sense.

Q4 Does the project ever need fully coupled simulations? If the answer is "no, we only run CLM against observed forcing data," then Option E (CTSM standalone) is arguably the most natural fit. If coupled experiments are on the roadmap, Options A-D preserve that capability.

Q5 Did the original ESCOMP container use a custom CTSM branch? Identifying exactly which CLM/CTSM the `escomp/cesm-lab-neon` image used would clarify the compatibility boundary.

CESM 3.x Release Timeline

For context on Option D: CESM 3.x was officially targeted for **spring 2026** (per the [CESM3 Plans page](#), last updated November 2025). As of June 2026, that target has slipped. The latest beta tag on GitHub (`beta08`) was published April 2, 2026, following a steady progression from `beta03` (Oct 2024) through `beta08` . The February 2026 CESM newsletter referenced "finalization and release of CESM3 later this year." No revised release date has been published. The release is tied to CMIP7 forcing data availability, an external dependency.

Best estimate: **late 2026**, but speculative. The project's NEON notebooks need to work now.

Recommendation

Option E (standalone CTSM) is recommended.

NCAR built the NEON workflow for standalone CTSM, not for CESM. Current CTSM releases have full NEON support available today. The multi-arch infrastructure, conda environment, and test framework from the rebuild carry over unchanged. The project runs land-only simulations with data atmosphere, which is exactly what standalone CTSM is designed for. The trade-off (losing coupled simulation capability) does not affect current workflows and can be revisited if scope expands.

See [ADR-0005](#) for the full decision record and implementation plan.